

List vs Dictionary in C# with Examples

In this article, I am going to discuss **List vs Dictionary in C#** with Examples. Please read our previous article where we discussed [Conversion Between Array List and Dictionary](#) in C#. At the end of this article, you will understand the difference between a List and Dictionary as well as you will understand when to use a List over Dictionary and vice-versa.

List vs Dictionary in C#

Both lists and dictionaries belong to Generics collections that are used to store collections of data. Both **Dictionary <TKey, TValue>** and **List <T>** are similar both have random access data structures on top of the .NET framework. The Dictionary is based on a hash table which means it uses a hash lookup, which is an efficient algorithm to look up things, on the other hand, a list, has to go and check element by element until it finds the result from the beginning. In this article, we will discuss List vs Dictionary in C#. When comparing with the List data structure, the dictionary always has a more or less fixed lookup time.

Let's go into the details.

The Dictionary uses the hashing algorithm to search for the element (data). A Dictionary first calculates a hash value for the key and this hash value leads to the target data bucket. After that, each element in the bucket needs to be checked for equality. But actually, the list will be faster than the dictionary on the first item search because nothing to search for in the first step. But in the second step, the list has to look through the first item and then the second item. So each step of the lookup takes more and more time. The larger the list, the longer it takes. Of course, the Dictionary in principle has a faster lookup with $O(1)$ while the lookup performance of a List is an $O(n)$ operation.

The Dictionary maps a key to a value and cannot have duplicate keys, whereas a list just contains a collection of values. Also, Lists allow duplicate items and support linear traversal.

Consider the following example:

```
Dictionary<string, int> dictionary = new Dictionary<string, int>();  
List<int> newList = new List<int>();
```

Add data to the list

```
newList.Add(data);
```

A list can simply add the item at the end of the existing list item. Add data to the Dictionary

```
dictionary.Add(key, data);
```

When you add data to a Dictionary, you should specify a unique key to the data so that it can be uniquely identified.

A Dictionary has a unique identifier, so whenever you look up a value in a Dictionary, the runtime must compute a hash code from the key. This optimized algorithm is implemented by some low-level bit shifting or modulo divisions. We determine the point at which Dictionary becomes more efficient for lookups than List.

Example to understand List vs Dictionary in C#:

The Find() method of the List class loops thru each object in the list until a match is found. So, if we want to look up a value using a key, then a dictionary is better for performance over the list. So, we need to use a dictionary when we know the collection will be primarily used for lookups.

```
using System;
using System.Collections.Generic;
namespace DictionaryVSListCollectionDemo
{
    public class Program
    {
        public static void Main()
        {
            Country country1 = new Country()
            {
                Code = "AUS",
                Name = "AUSTRALIA",
                Capital = "Canberra"
            };
            Country country2 = new Country()
            {
                Code = "IND",
                Name = "INDIA ",
                Capital = "New Delhi"
            };
            Country country3 = new Country()
            {
                Code = "USA",
                Name = "UNITED STATES",
                Capital = "Washington D.C."
            };
            Country country4 = new Country()
            {
                Code = "GBR",
                Name = "UNITED KINGDOM",
                Capital = "London"
            };
            Country country5 = new Country()
            {
                Code = "CAN",
                Name = "CANADA",
                Capital = "Ottawa"
            };
            //List<Country> listCountries = new List<Country>();
            //listCountries.Add(country1);
            //listCountries.Add(country2);
            //listCountries.Add(country3);
            //listCountries.Add(country4);
            //listCountries.Add(country5);
            Dictionary<string, Country> dictionaryCountries = new Dictionary<
Country>();
            dictionaryCountries.Add(country1.Code, country1);
```

```

dictionaryCountries.Add(country2.Code, country2);
dictionaryCountries.Add(country3.Code, country3);
dictionaryCountries.Add(country4.Code, country4);
dictionaryCountries.Add(country5.Code, country5);
string strUserChoice = string.Empty;
do
{
    Console.WriteLine("Please enter country code");
    string strCountryCode = Console.ReadLine().ToUpper();
    // Find() method of the list class loops thru each object in list
    // until a match is found. So, if we want to lookup a value using a key dictionary is better for performance.
    // Country resultCountry = listCountries. Find(country => country.Code == strCountryCode);
    Country resultCountry = dictionaryCountries.ContainsKey(strCountryCode) ? dictionaryCountries[strCountryCode] : null;
    if (resultCountry == null)
    {
        Console.WriteLine("The country code you entered does not exist");
    }
    else
    {
        Console.WriteLine("Name = " + resultCountry.Name + " Capital = " + resultCountry.Capital);
    }
    do
    {
        Console.WriteLine("Do you want to continue - YES or NO?");
        strUserChoice = Console.ReadLine().ToUpper();
    }
    while (strUserChoice != "NO" && strUserChoice != "YES");
}
while (strUserChoice == "YES");
}
}

public class Country
{
    public string Name { get; set; }
    public string Code { get; set; }
    public string Capital { get; set; }
}
}

```

Output:

```
Please enter country code
IND
Name = INDIA Captial =New Delhi
Do you want to continue - YES or NO?
YES
Please enter country code
AUS
Name = AUSTRALIA Captial =Canberra
Do you want to continue - YES or NO?
NO
```

In the next article, I am going to discuss [Generic Stack Collection Class in C#](#) with Examples. Here, in this article, I try to explain **List vs Dictionary in C#** with an example. I hope this article will help you with your need. I would like to have your feedback. Please post your feedback, question, or comments about this article.